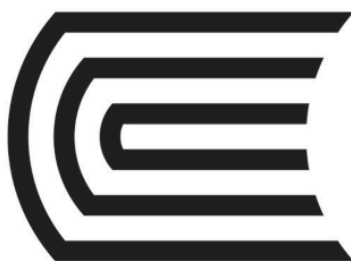


**“ Año de la Esperanza y el Fortalecimiento de la
Democracia ”**



**Universidad
Continental**

Asignatura:
ESTRUCTURA DE DATOS

NRC: 28649

Nro Grupo: 3

Proyecto Grupal
“Desarrollar un Sistema de Gestión de Procesos”

- Integrantes:**
1. Malpartida Arroyo Joseph Rafael
 2. Ñaupari Bonilla Miguel Fabricio
 3. Celis Ricra Daniel Angel
 4. Barrientos Chambergo Miguel Ángel

PERÚ
HUANCAYO - 2026

ÍNDICE

Capítulo 1	3
Análisis del problema	3
1. Descripción del problema	4
2. Requerimientos del sistema	5
2.1. Requerimientos Funcionales	5
2.2. Requerimientos No Funcionales	6
3. Estructuras de datos propuestas	7
4. Justificación de la elección:	8
4.1. Lista Enlazada Simple “Gestor de Procesos”	8
4.2. Cola de Prioridad “Planificador de CPU”	8
4.3. Pila “Gestor de Memoria”	9
4.4. Uso de Punteros, Funciones y Estructuras de Control	9
Capítulo 2	10
Diseño de la Solución	10
1. Descripción de estructuras de datos y operaciones:	11
2. Algoritmos principales:	13
3. Diagramas de Flujo:	28
4. Justificación del diseño:	30
Capítulo 3	31
Solución Final	31
1. Código limpio, bien comentado y estructurado.	32
2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación de datos	32
3. Manual de usuario	32
Capítulo 4	33
Evidencias de trabajo colaborativo	33
1. Repositorio y control de versiones	34
1.1. Registro de commits	34
1.2. Historial de ramas	35
1.3. Evidencia por cada integrante del equipo	35
1.3.1. Malpartida Arroyo Joseph Rafael	35
1.3.2. Ñaupari Bonilla Miguel Fabricio	35
1.3.3. Celis Ricra Daniel Angel	36
1.3.4. Barrientos Chambero Miguel A	36
1.4. Enlace a la herramienta colaborativa	37
1.5. Enlace a las diapositivas de exposicion	37
2. Plan de trabajo y roles + asignados	37
2.1. Documento inicial donde se asignan tareas y responsabilidades.	37
2.2. Cronograma con fechas límite para cada entrega parcial.	38
2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).	38

Capítulo 1

Análisis del problema

1. Descripción del problema

En la actualidad los sistemas operativos modernos deben administrar múltiples procesos de manera simultánea para garantizar que el sistema funcione de forma eficiente, estable y organizada. Durante la ejecución del sistema la información cambia constantemente debido a que algunos procesos son creados, otros finalizan su ejecución y varios requieren diferentes niveles de prioridad para acceder a los recursos disponibles. Como consecuencia, la administración adecuada de estos procesos se convierte en una tarea fundamental para mantener un rendimiento aceptable y evitar conflictos durante el funcionamiento del sistema.

Cuando no existe una gestión adecuada de los procesos pueden presentarse diversos problemas como tiempos de espera elevados, utilización ineficiente del procesador, dificultades para localizar información específica y un uso desorganizado de la memoria. Además, la ausencia de mecanismos adecuados para almacenar y actualizar información puede afectar el desempeño general del sistema y dificultar el control de los recursos disponibles.

Con la finalidad de comprender mejor cómo se administran estos elementos dentro de un entorno computacional, se plantea el desarrollo de un Sistema de Gestión de Procesos (SGP) implementado en C++. Este sistema simulará algunas funciones básicas que realiza un sistema operativo mediante el uso de estructuras de datos implementadas manualmente con nodos y punteros. El objetivo principal es representar de manera práctica cómo se organizan los procesos, cómo se determina su orden de ejecución y cómo se administran determinados recursos durante la operación del sistema.

Para resolver las necesidades identificadas, el proyecto estará organizado en diferentes módulos especializados. El gestor de procesos se encargará del registro y mantenimiento de la información de cada proceso. El planificador de CPU administrará el orden en que los procesos serán ejecutados según su prioridad. El gestor de memoria permitirá representar la asignación y liberación de recursos asociados a los procesos. Adicionalmente, se incorporará un módulo de búsqueda indexada orientado a mejorar la velocidad de localización de información mediante mecanismos de acceso directo.

La integración de estos módulos permitirá desarrollar una herramienta educativa capaz de representar conceptos fundamentales relacionados con la administración de procesos y el uso de estructuras de datos dentro de un sistema computacional. Asimismo, el proyecto servirá para reforzar conocimientos sobre memoria dinámica, organización de información y diseño modular de programas.

2. Requerimientos del sistema

Los requerimientos del sistema se dividen en funcionales y no funcionales, los funcionales son todas las acciones y operaciones que realizará el sistema durante su ejecución, mientras que los no funcionales son las condiciones y características técnicas que el programa deberá cumplir para funcionar de manera correcta, ordenada y estable durante su uso.

2.1. Requerimientos Funcionales

Los requerimientos funcionales fueron organizados según cada módulo del sistema, ya que cada uno cumple una función diferente dentro del programa.

2.1.1. Módulo Gestor de Procesos

- RF-01 El sistema permitirá registrar nuevos procesos almacenando datos como el ID, nombre, prioridad y estado de cada proceso.
- RF-02 También permitirá eliminar procesos cuando ya no sean necesarios dentro del sistema.
- RF-03 El usuario podrá buscar procesos mediante su ID o nombre para visualizar toda su información registrada.
- RF-04 Será posible modificar la prioridad de un proceso para reorganizar su importancia dentro del sistema.
- RF-05 Además, el sistema mostrará la lista completa de procesos registrados junto con todos sus datos correspondientes.

2.1.2. Módulo Planificador de CPU

- RF-06 El sistema organizará los procesos dentro de una cola según su nivel de prioridad para mantener un mejor orden de ejecución.
- RF-07 También ejecutará procesos mediante operaciones de desencholamiento respetando el orden establecido dentro de la cola.
- RF-08 El usuario podrá visualizar el estado actual de la cola de procesos y el orden en que serán atendidos.

2.1.3. Módulo Gestor de Memoria

- RF-09 El sistema permitirá asignar bloques de memoria utilizando operaciones push sobre una pila.
- RF-10 Además, será posible liberar memoria eliminando el último bloque asignado mediante operaciones pop.

- RF-11 El usuario podrá visualizar el estado actual de la memoria y los bloques almacenados dentro de la pila.

2.1.4. Interfaz y Persistencia

- RF-12 El sistema contará con un menú interactivo por consola para facilitar el acceso a todas las opciones disponibles.
- RF-13 El programa validará los datos ingresados por el usuario mostrando mensajes de error cuando exista información inválida.
- RF-14 El sistema permitirá guardar la información registrada en archivos de texto para conservar los datos durante diferentes sesiones.
- RF-15 También podrá recuperar la información almacenada cuando el programa vuelva a ejecutarse.

2.2. Requerimientos No Funcionales

Los requerimientos no funcionales representan las condiciones técnicas y características de calidad que el sistema deberá cumplir durante su desarrollo y funcionamiento.

- RNF-01 El proyecto será desarrollado utilizando DEV C++ en un entorno Windows.
- RNF-02 Solo se utilizarán estructuras de datos dinámicas lineales para el desarrollo del sistema.
- RNF-03 Las estructuras deberán implementarse manualmente mediante nodos y punteros, sin utilizar librerías predefinidas.
- RNF-04 Cada estructura incluirá operaciones fundamentales como inserción, eliminación, búsqueda y recorrido de información.
- RNF-05 El código fuente deberá mantenerse organizado mediante funciones y módulos para facilitar su comprensión y mantenimiento.
- RNF-06 El sistema contará con almacenamiento permanente de datos mediante archivos de texto.
- RNF-07 El menú interactivo deberá ser claro, sencillo y capaz de recuperarse ante entradas inválidas sin cerrar el programa.
- RNF-08 El código fuente será almacenado dentro de un repositorio Git para mantener el control de versiones del proyecto.

- RNF-09 El proyecto incluirá documentación técnica en formato PDF explicando el funcionamiento del sistema.
- RNF-10 El sistema deberá ejecutarse correctamente durante las pruebas y la demostración final del proyecto.

3. Estructuras de datos propuestas

Para el desarrollo del sistema se utilizarán diferentes estructuras de datos debido a que cada módulo presenta necesidades particulares relacionadas con el almacenamiento, acceso y procesamiento de información. La selección de estas estructuras busca aprovechar las características específicas de cada una para facilitar la implementación de las operaciones requeridas por el sistema.

3.1. Lista Enlazada Simple “Gestor de Procesos”

La lista enlazada simple será utilizada para almacenar y administrar la información correspondiente a los procesos registrados dentro del sistema. Esta estructura permitirá mantener una colección dinámica de elementos cuyo tamaño podrá variar durante la ejecución del programa. Gracias a su organización mediante nodos enlazados, será posible incorporar nuevos procesos, eliminar registros existentes y actualizar información sin depender de posiciones fijas de memoria.

3.2. Cola de Prioridad “Planificador de CPU”

La cola de prioridad será utilizada para organizar el orden de ejecución de los procesos. Su funcionamiento permitirá que los procesos con mayor nivel de prioridad reciban atención antes que aquellos menos importantes, facilitando la simulación de un mecanismo básico de planificación de CPU. De esta forma se podrá representar cómo un sistema operativo decide qué tarea ejecutar en cada momento.

3.3. Pila “Gestor de Memoria”

La pila será empleada para representar la administración de bloques de memoria asociados a los procesos. Su comportamiento basado en el principio de último en entrar y primero en salir permitirá modelar de manera simplificada la asignación y liberación secuencial de recursos durante la ejecución del sistema.

3.4. Tabla Hash “Módulo de Búsqueda Indexada (MBI)”

La tabla hash será utilizada para optimizar la localización de procesos registrados dentro del sistema. Mediante una función de dispersión aplicada sobre el identificador de cada proceso será posible acceder a la información de manera más rápida que mediante recorridos secuenciales completos. Esto permitirá mejorar el tiempo de respuesta de determinadas consultas realizadas por el usuario.

4. Justificación de la elección:

Las estructuras seleccionadas fueron elegidas considerando las funciones específicas que deberá cumplir cada módulo dentro del Sistema de Gestión de Procesos. Cada una aporta características particulares que permiten administrar la información de manera más eficiente y representar diferentes mecanismos relacionados con la organización de procesos y recursos dentro de un entorno computacional.

4.1. Lista Enlazada Simple “Gestor de Procesos”

La lista enlazada simple se utilizará para organizar los procesos porque permite trabajar con información de una manera flexible, ya que la cantidad de procesos dentro del sistema puede cambiar constantemente durante la ejecución del programa. Gracias a esta estructura será posible agregar nuevos procesos, eliminar procesos existentes, buscar información específica o modificar datos sin necesidad de reorganizar completamente la información almacenada.

Además, debido a que los nodos se encuentran conectados mediante punteros, el sistema podrá administrar la memoria de forma dinámica y adaptarse a diferentes cantidades de procesos sin depender de un tamaño fijo establecido desde el inicio. Esto permitirá mantener un registro actualizado y organizado de todos los procesos existentes.

4.2. Cola de Prioridad “Planificador de CPU”

La cola de prioridad fue elegida porque permite representar de manera más adecuada el funcionamiento de un planificador de CPU. A diferencia de una cola convencional, los procesos no serán atendidos únicamente por orden de llegada, sino también considerando su nivel de prioridad dentro del sistema.

Gracias a esta característica, los procesos más importantes podrán ejecutarse primero, mejorando la organización de las tareas y simulando de una manera más cercana el comportamiento básico utilizado por los sistemas operativos para administrar la ejecución de procesos. Asimismo, esta estructura facilita el control del orden de atención y la actualización de prioridades cuando sea necesario.

4.3. Pila “Gestor de Memoria”

La pila se utilizará para representar la administración de memoria debido a que funciona bajo el principio de “último en entrar, primero en salir”. Este

comportamiento permite modelar de manera sencilla la asignación y liberación secuencial de recursos durante la ejecución del sistema.

Mediante operaciones push será posible registrar nuevos bloques de memoria, mientras que las operaciones pop permitirán liberar el último bloque asignado. Esto contribuirá a mantener un mejor control sobre los recursos utilizados y a representar de forma práctica el funcionamiento de determinadas operaciones relacionadas con la gestión de memoria.

4.4. Tabla Hash “Módulo de Búsqueda Indexada”

La tabla hash fue incorporada para optimizar los procesos de búsqueda dentro del sistema. Su principal ventaja consiste en permitir el acceso rápido a la información mediante el uso de una función de dispersión que transforma el identificador de un proceso en una posición específica dentro de la estructura.

Gracias a este mecanismo será posible localizar procesos de manera más eficiente que realizando recorridos completos sobre la lista principal. Esto resulta especialmente útil cuando la cantidad de procesos registrados aumenta, ya que mejora los tiempos de respuesta y facilita las consultas frecuentes realizadas por el usuario.

4.5. Uso de Punteros, Funciones y Estructuras de Control

Los punteros serán fundamentales dentro del proyecto porque permitirán conectar nodos y administrar memoria dinámica de manera eficiente. Su utilización facilitará la creación, modificación, recorrido y eliminación de elementos dentro de las diferentes estructuras implementadas.

Además, el programa será desarrollado utilizando funciones con el propósito de mantener un código más organizado, modular y fácil de comprender. La división del sistema en funciones independientes permitirá reutilizar procedimientos, simplificar el mantenimiento del código y facilitar futuras mejoras.

También se emplearán diversas estructuras de control según las necesidades de cada módulo. El ciclo while permitirá recorrer listas, colas y pilas mientras existan nodos disponibles. El ciclo do while será utilizado principalmente para controlar el menú principal garantizando que este se ejecute al menos una vez. Por su parte, el ciclo for se empleará en tareas donde sea necesario repetir instrucciones una cantidad determinada de veces.

De igual manera, la estructura switch facilitará la administración de las opciones disponibles dentro del menú interactivo, mientras que las condiciones if e if-else permitirán validar datos, verificar estados y controlar el flujo lógico de ejecución del sistema.

Capítulo 2

Diseño de la Solución

1. Descripción de estructuras de datos y operaciones:

El diseño interno del Sistema de Gestión de Procesos se basa en la implementación manual de estructuras de datos mediante nodos y punteros. Cada módulo posee una estructura específica encargada de almacenar información, ejecutar operaciones y mantener la organización de los datos durante el funcionamiento del programa. En esta sección se describe la composición de las estructuras y las operaciones que permiten manipular la información dentro del sistema.

1.1. Lista Enlazada Simple “Módulo Gestor de Procesos”

La lista enlazada simple constituye la estructura principal para el almacenamiento de procesos. Cada proceso se representa mediante un nodo independiente conectado con el siguiente elemento de la estructura a través de un puntero. La información permanece organizada de forma secuencial, permitiendo recorrer todos los registros desde el primer nodo hasta el último.

Durante el funcionamiento del sistema, esta estructura actúa como repositorio principal de información, ya que mantiene los datos necesarios para las operaciones de consulta, actualización y administración de procesos. El recorrido de la lista se realiza siguiendo los enlaces establecidos entre los nodos hasta alcanzar el final de la estructura.

Composición del nodo:

- id: identificador único del proceso dentro del sistema.
- nombre: nombre que identifica al proceso.
- prioridad: nivel de importancia asignado al proceso.
- estado: indica si el proceso está activo, en espera o ya finalizado.
- siguiente: puntero que conecta con el siguiente nodo de la lista.

Operaciones implementadas:

- Inserción de procesos: permite registrar nuevos procesos dentro del sistema, creando un nodo nuevo y luego enlazándolo con la lista existente.
- Eliminación de procesos: permite buscar un proceso por su ID y eliminarlo, ajustando los enlaces correspondientes para no perder la continuidad de la lista.
- Búsqueda de procesos: permite recorrer la lista completa hasta encontrar un proceso por ID o por nombre y mostrar su información.

- Modificación de prioridad: permite cambiar el nivel de prioridad de un proceso ya registrado para reorganizar su importancia dentro del sistema.
- Cambio de estado: permite actualizar la condición del proceso dependiendo de su situación durante la ejecución.
- Listado de procesos: permite recorrer toda la lista y mostrar todos los procesos almacenados con su información completa.

1.2. Cola de Prioridad “Módulo Planificador de CPU”

La cola de prioridad administra los procesos pendientes de ejecución. La organización interna considera el nivel de prioridad de cada elemento, permitiendo que determinados procesos ocupen posiciones preferenciales dentro de la estructura. De esta manera, el sistema mantiene un orden de atención coherente con los criterios definidos para la planificación.

Cada vez que un proceso ingresa a la cola, su ubicación es determinada considerando la prioridad registrada. El frente de la estructura representa el siguiente proceso que será atendido por el planificador.

Composición del nodo

- idProceso: identificador del proceso.
- nombre: nombre del proceso.
- prioridad: valor que determina el orden de ejecución dentro de la cola.
- estadoEjecucion: indica el estado actual del proceso dentro del sistema.
- siguiente: puntero que conecta con el siguiente nodo de la cola.

Operaciones implementadas

- Encolamiento: permite agregar procesos a la cola, ubicándolos en la posición correcta según su prioridad.
- Desencolamiento: permite simular la ejecución del proceso eliminando el elemento que se encuentra al frente de la cola.
- Visualización de cola: permite mostrar todos los procesos que están esperando su turno de ejecución.
- Reorganización por prioridad: permite ordenar nuevamente la cola dependiendo del nivel de importancia de los procesos.

1.3. Pila “Módulo Gestor de Memoria”

La pila mantiene el registro de los bloques de memoria asignados durante la ejecución del sistema. Su funcionamiento se centra en la parte superior de la estructura, donde se realizan todas las operaciones de inserción y eliminación.

La administración mediante pila permite controlar de forma ordenada los recursos almacenados y facilita la representación del comportamiento secuencial utilizado en determinados mecanismos de gestión de memoria. El acceso directo al elemento superior simplifica las operaciones de asignación y liberación de recursos.

Composición del nodo

- idProceso: identificador del proceso al que pertenece el bloque de memoria.
- bloqueMemoria: referencia al bloque asignado dentro del sistema.
- tamanoBloque: tamaño del bloque de memoria utilizado.
- siguiente: puntero que conecta con el siguiente nodo de la pila.

Operaciones implementadas

- Push (asignación de memoria): permite agregar nuevos bloques de memoria a la pila, colocándolos en la parte superior.
- Pop (liberación de memoria): permite eliminar el último bloque agregado y liberar el espacio correspondiente.
- Visualización de memoria: permite mostrar todos los bloques actualmente almacenados en la pila.
- Consulta de memoria: permite revisar el estado general de la memoria utilizada dentro del sistema.

1.4. Tabla Hash “Módulo de Búsqueda Indexada”

La tabla hash complementa las demás estructuras permitiendo localizar procesos de forma más rápida mediante acceso indexado. Su funcionamiento se basa en una función matemática que transforma el identificador de un proceso en una posición específica dentro de un arreglo.

Cuando se produce una consulta, el sistema calcula el índice correspondiente y accede directamente a la ubicación donde debería encontrarse la información. En caso de colisiones, se

aplica exploración lineal para localizar una posición disponible o continuar la búsqueda.

Operaciones implementadas

- Cálculo hash: determina el índice asociado a cada identificador.
- Inserción indexada: registra referencias de procesos dentro de la posición calculada.
- Búsqueda indexada: localiza procesos utilizando acceso directo mediante índices.
- Eliminación lógica: libera posiciones ocupadas o marca registros eliminados para mantener la consistencia de la estructura.

2. Algoritmos principales:

Los siguientes algoritmos representan las operaciones principales del Sistema de Gestión de Procesos, y se organizan por módulos para que todo esté más ordenado dentro del sistema y también para entender mejor cómo funciona el programa en general, ya que cada módulo hace una función distinta dentro de la simulación del sistema operativo, y así se puede ver mejor cómo se va ejecutando todo durante el proceso.

2.1. Algoritmo para insertar un nuevo proceso en el sistema

Proceso insertarProceso

Definir idBuscado, nuevold, nuevoNombre Como Cadena

Definir nuevaPrioridad Como Entero

Definir encontrado Como Logico

Definir i Como Entero

Dimension id[100], nombre[100], prioridad[100]

Dimension estado[100]

Escribir "Ingrese ID del nuevo proceso: "

Leer nuevold

encontrado <- Falso

i <- 1

Mientras i <= 100 Y id[i] <> "" Hacer

Si id[i] = nuevold Entonces

encontrado <- Verdadero

FinSi

i <- i + 1

FinMientras

Si encontrado = Verdadero Entonces

Escribir "Error: ya existe un proceso con ese ID"

SiNo

i <- 1

Mientras id[i] <> "" Hacer

i <- i + 1

FinMientras

id[i] <- nuevold

Escribir "Ingrese nombre del proceso: "

Leer nuevoNombre

nombre[i] <- nuevoNombre

Escribir "Ingrese prioridad: "

Leer nuevaPrioridad

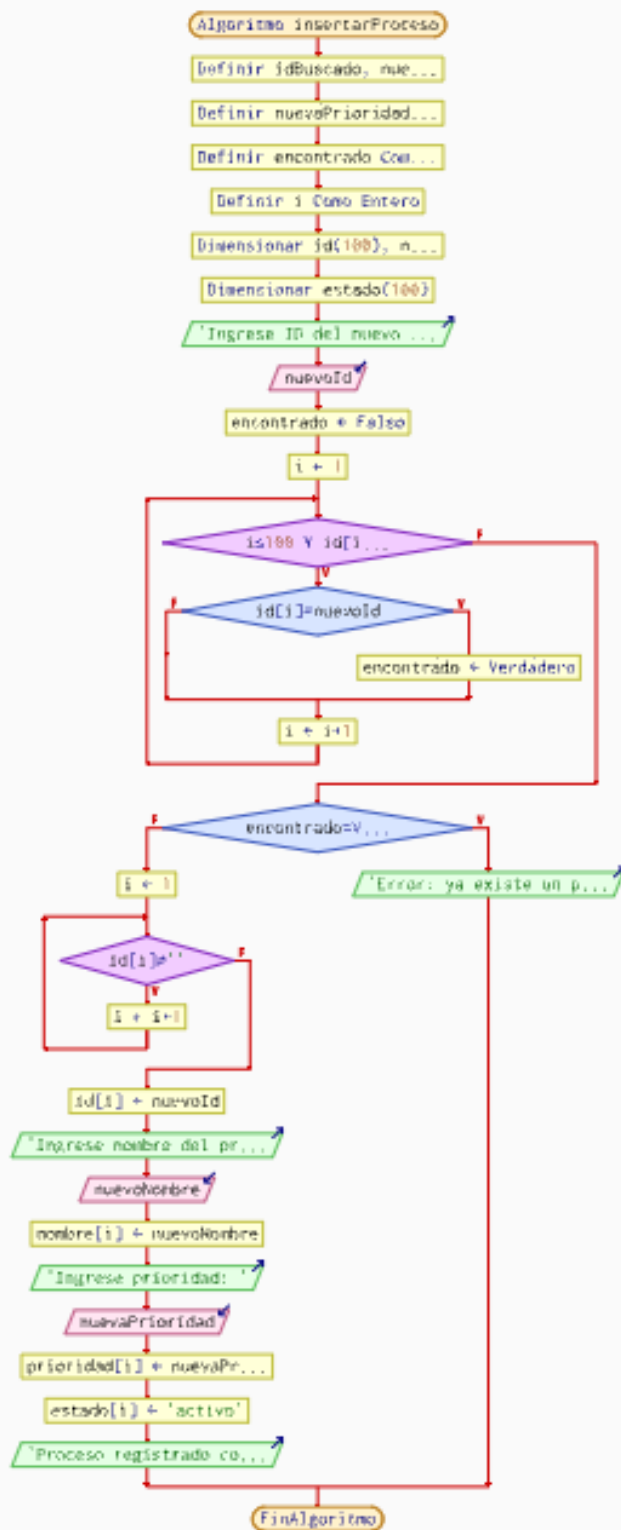
prioridad[i] <- nuevaPrioridad

estado[i] <- "activo"

Escribir "Proceso registrado correctamente"

FinSi

FinProceso



2.2. Algoritmo para eliminar un proceso por ID o nombre

Proceso eliminarProceso

Definir criterio, valorBuscado Como Cadena

Definir i Como Entero

Definir encontrado Como Logico

Dimension id[100], nombre[100], prioridad[100], estado[100]

Escribir "Buscar por ID o Nombre: "

Leer criterio

Escribir "Ingrese valor a buscar: "

Leer valorBuscado

encontrado <- Falso

i <- 1

Mientras i <= 100 Y id[i] <> "" Hacer

Si (criterio = "ID" Y id[i] = valorBuscado) O (criterio = "Nombre" Y
nombre[i] = valorBuscado) Entonces

id[i] <- ""

nombre[i] <- ""

prioridad[i] <- 0

estado[i] <- ""

encontrado <- Verdadero

FinSi

i <- i + 1

FinMientras

Si encontrado = Verdadero Entonces

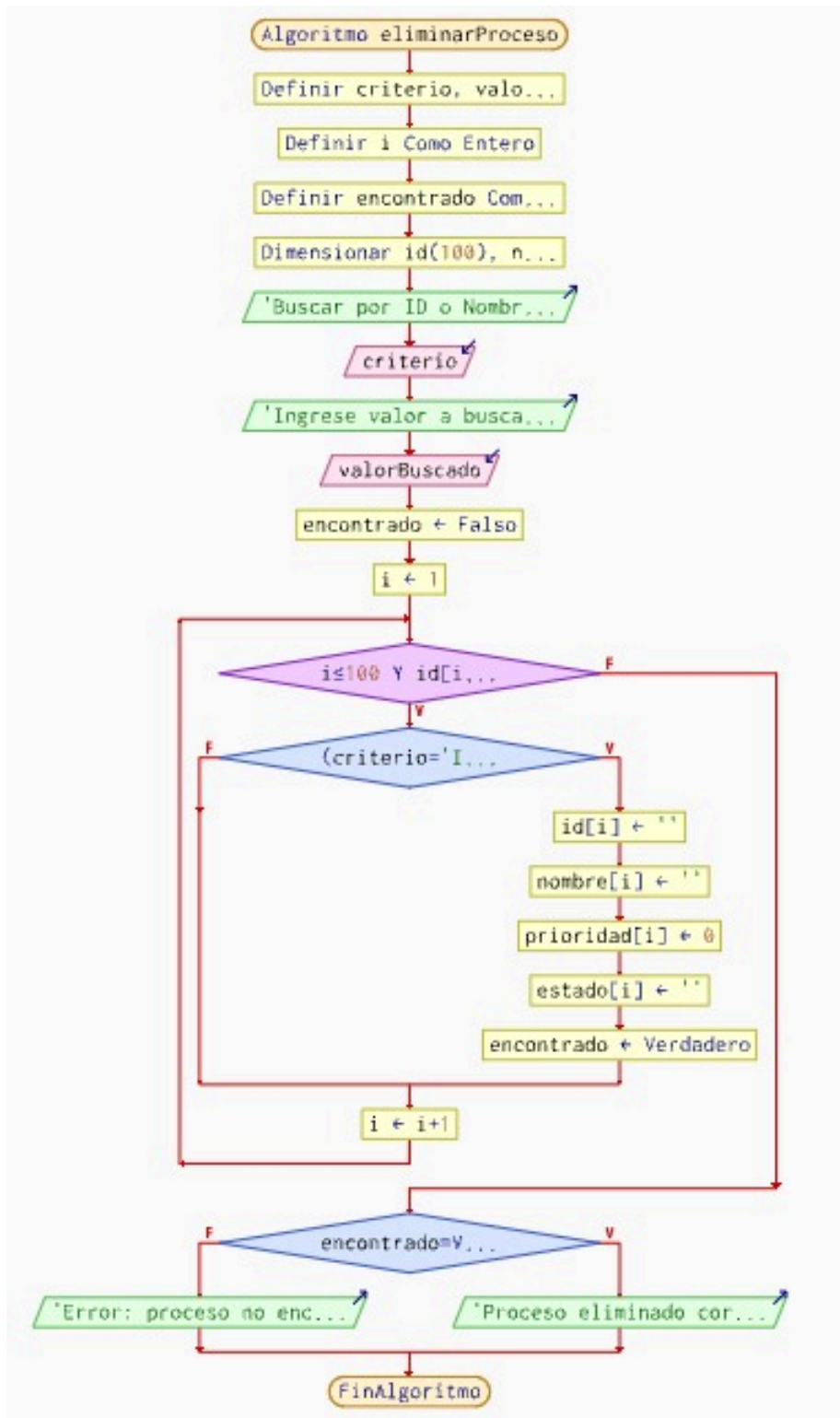
Escribir "Proceso eliminado correctamente"

SiNo

Escribir "Error: proceso no encontrado"

FinSi

FinProceso



2.3. Algoritmo para buscar un proceso dentro del sistema

Proceso buscarProceso

Definir criterio, valorBuscado Como Cadena

Definir i Como Entero

Definir encontrado Como Logico

Dimension id[100], nombre[100], prioridad[100], estado[100]

Escribir "Buscar por ID o Nombre: "

Leer criterio

Escribir "Ingrese valor a buscar: "

Leer valorBuscado

encontrado <- Falso

i <- 1

Mientras i <= 100 Y id[i] <> "" Hacer

Si (criterio = "ID" Y id[i] = valorBuscado) O (criterio = "Nombre" Y
nombre[i] = valorBuscado) Entonces

Escribir "=== Proceso encontrado ==="

Escribir "ID: ", id[i]

Escribir "Nombre: ", nombre[i]

Escribir "Prioridad: ", prioridad[i]

Escribir "Estado: ", estado[i]

encontrado <- Verdadero

FinSi

i <- i + 1

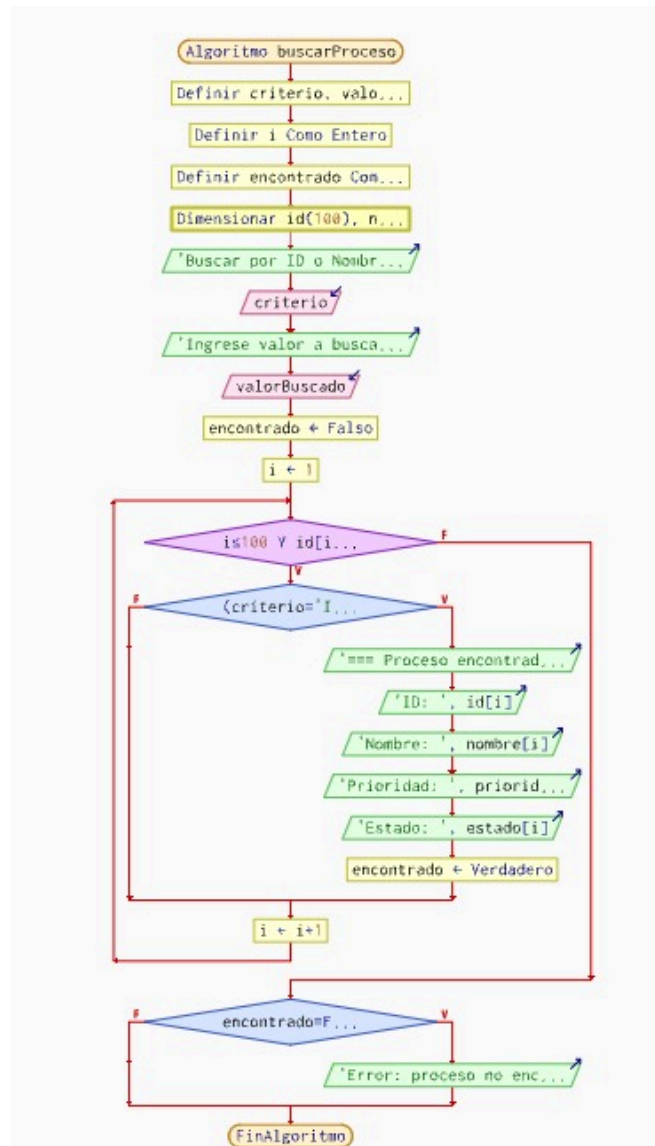
FinMientras

Si encontrado = Falso Entonces

Escribir "Error: proceso no encontrado"

FinSi

FinProceso



2.4. Algoritmo para modificar la prioridad de un proceso

Proceso ModificarPrioridad

Definir idBuscado, nuevaPrioridad, i, n Como Entero

Definir encontrado Como Logico

Dimension ids[100]

Dimension prioridades[100]

encontrado <- FALSO

Escribir "Ingrese ID a modificar:"

Leer idBuscado

Escribir "Ingrese nueva prioridad (1-5):"

Leer nuevaPrioridad

Si nuevaPrioridad < 1 O nuevaPrioridad > 5 Entonces

Escribir "Error: prioridad inválida"

SiNo

Para i <- 0 Hasta n-1 Hacer

Si ids[i] = idBuscado Entonces

prioridades[i] <- nuevaPrioridad

encontrado <- VERDADERO

Escribir "Prioridad modificada"

FinSi

FinPara

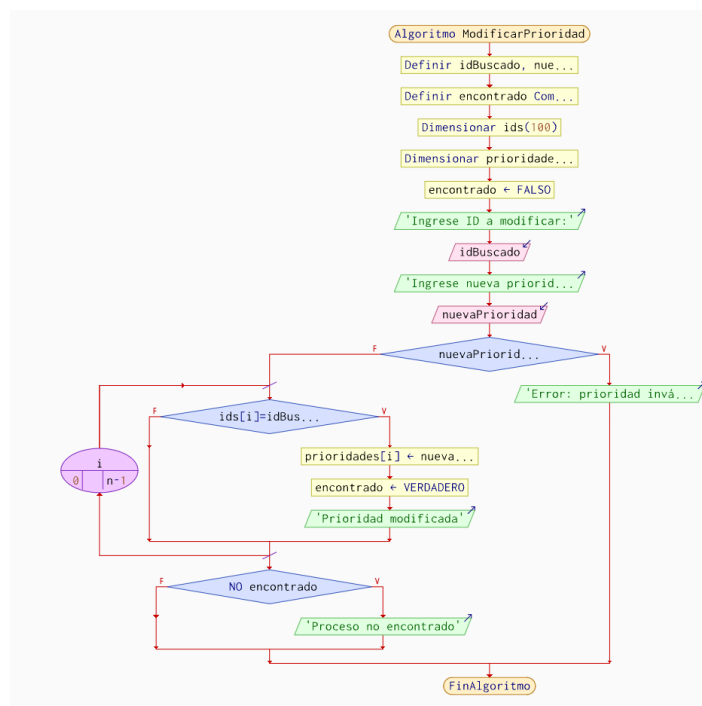
Si NO encontrado Entonces

Escribir "Proceso no encontrado"

FinSi

FinSi

FinProceso



2.5. Algoritmo para listar todos los procesos registrados

Proceso ListarProcesos

Definir i, n Como Entero

Dimensionar ids[100]

Dimensionar nombres[100]

Dimensionar prioridades[100]

Si $n = 0$ Entonces

 Escribir "No hay procesos"

SiNo

 Para $i \leftarrow 0$ Hasta $n-1$ Hacer

 Escribir "ID: ", ids[i]

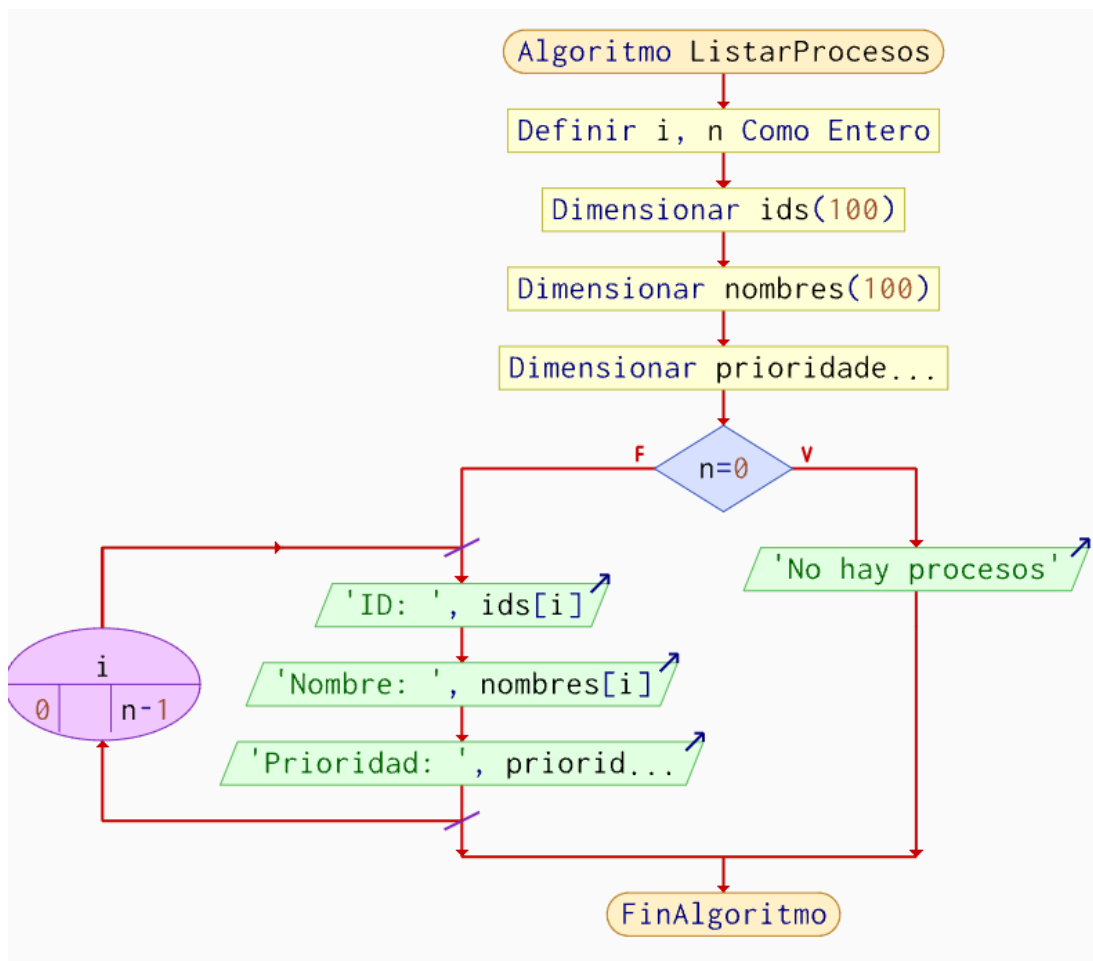
 Escribir "Nombre: ", nombres[i]

 Escribir "Prioridad: ", prioridades[i]

 FinPara

FinSi

FinProceso



2.6. Algoritmo para encolar procesos según prioridad en la CPU

Proceso EncolarPrioridad

Definir id, prioridad, i, n Como Entero

Definir nombre Como Cadena

Dimension ids[100], nombres[100], prioridades[100]

Escribir "ID:"; Leer id

Escribir "Nombre:"; Leer nombre

Escribir "Prioridad:"; Leer prioridad

$i \leftarrow n$

Mientras $i > 0$ Y prioridades[i-1] < prioridad Hacer

ids[i] <- ids[i-1]

nombres[i] <- nombres[i-1]

prioridades[i] <- prioridades[i-1]

$i \leftarrow i - 1$

FinMientras

ids[i] <- id

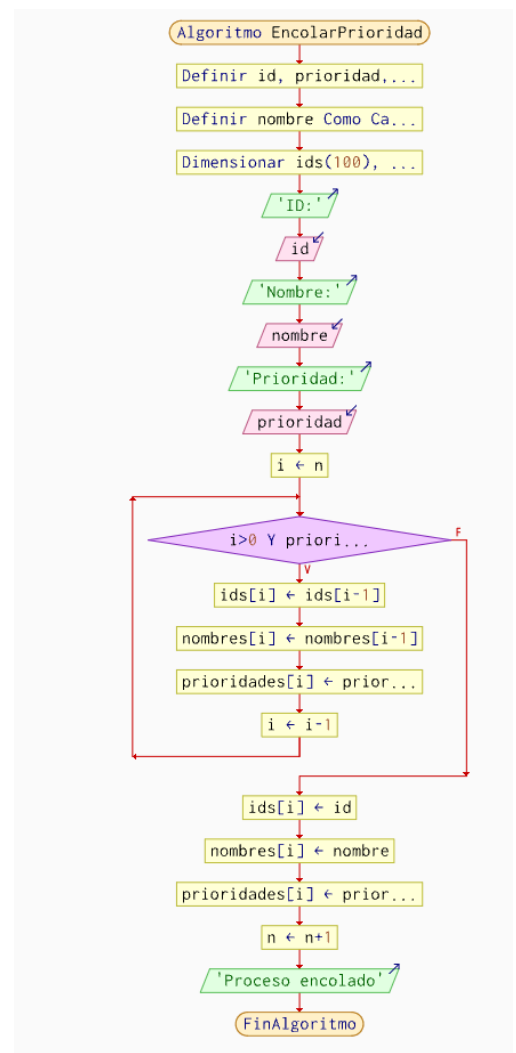
nombres[i] <- nombre

prioridades[i] <- prioridad

$n \leftarrow n + 1$

Escribir "Proceso encolado"

FinProceso



2.7. Algoritmo para desencolar y ejecutar un proceso

Proceso desencolarEjecutarProceso

Definir frente, final, i Como Entero

Definir idProceso, prioridadProceso Como Entero

Definir nombreProceso Como Cadena

Dimensionar id[100], nombre[100], prioridad[100]

Si frente = -1 Entonces

Escribir "Cola vacia"

SiNo

idProceso <- id[frente]

nombreProceso <- nombre[frente]

prioridadProceso <- prioridad[frente]

Escribir "Ejecutando proceso"

Para i <- frente Hasta final - 1 Hacer

id[i] <- id[i+1]

nombre[i] <- nombre[i+1]

prioridad[i] <- prioridad[i+1]

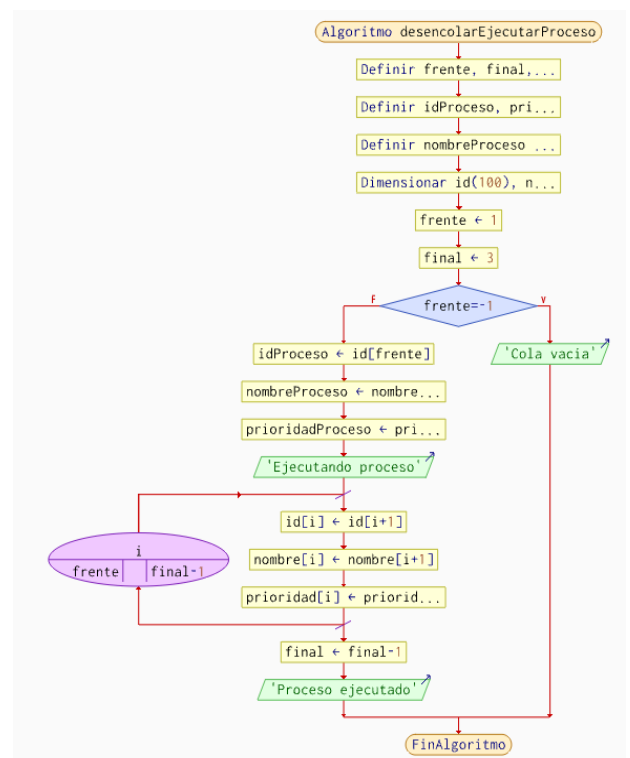
FinPara

final <- final - 1

Escribir "Proceso ejecutado"

FinSi

FinProceso



2.8. Algoritmo para visualizar la cola de procesos de la CPU

Proceso visualizarColaCPU

Definir frente, final, i Como Entero

Definir nombre Como Cadena

Dimension nombres[100]

Si frente = -1 Entonces

Escribir "Cola vacia"

SiNo

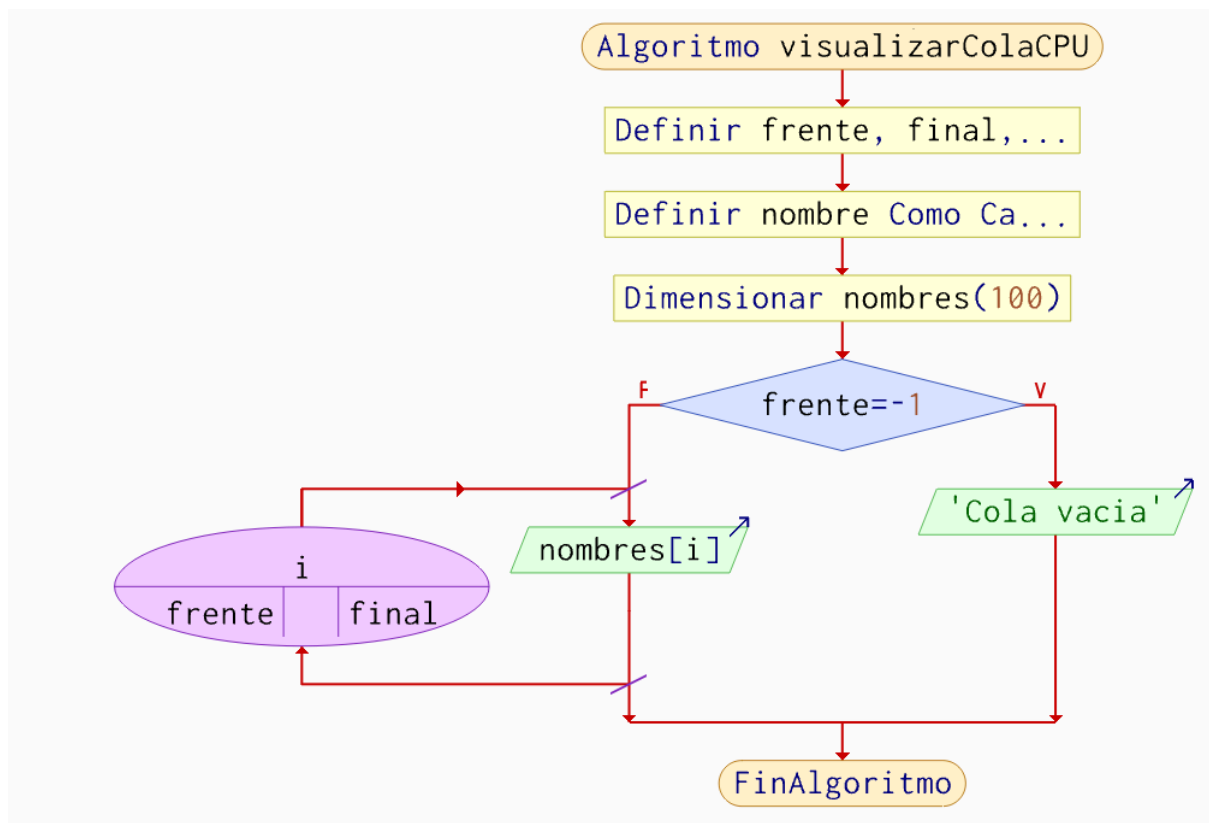
Para i <- frente Hasta final Hacer

Escribir nombres[i]

FinPara

FinSi

FinProceso



2.9. Algoritmo para asignación de memoria (Push)

Proceso asignarMemoriaPush

Definir tope, memoria Como Entero

Dimensionar pila[100]

Si tope = 99 Entonces

 Escribir "Memoria llena"

SiNo

 Escribir "Ingrese memoria:"

 Leer memoria

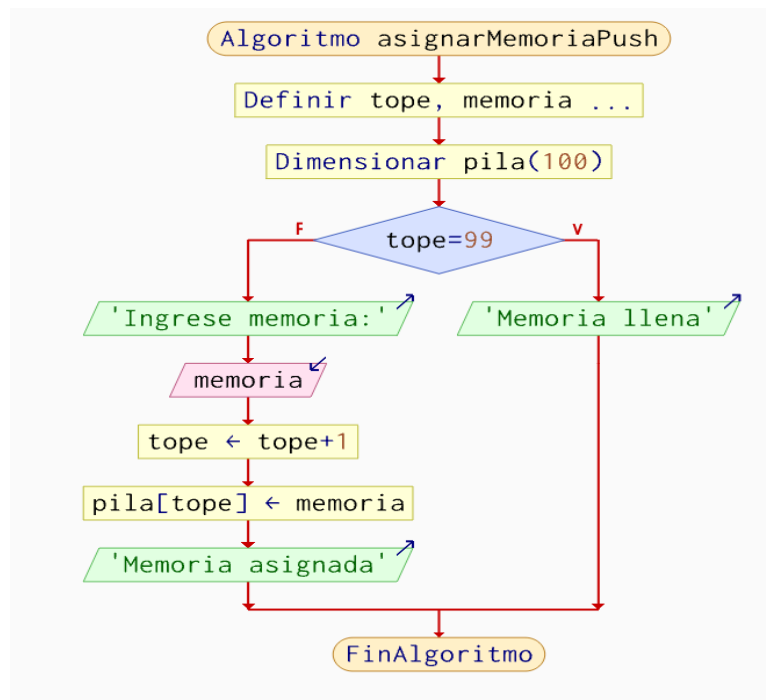
 tope $\leftarrow$ tope + 1

 pila[tope] $\leftarrow$ memoria

 Escribir "Memoria asignada"

FinSi

FinProceso



2.10. Algoritmo para liberación de memoria (Pop)

Algoritmo LiberarMemoriaPop

Definir pila[100] Como Entero

Definir tope, dato Como Entero

Si tope = -1 Entonces

 Escribir "La pila está vacía"

Sino

 dato <- pila[tope]

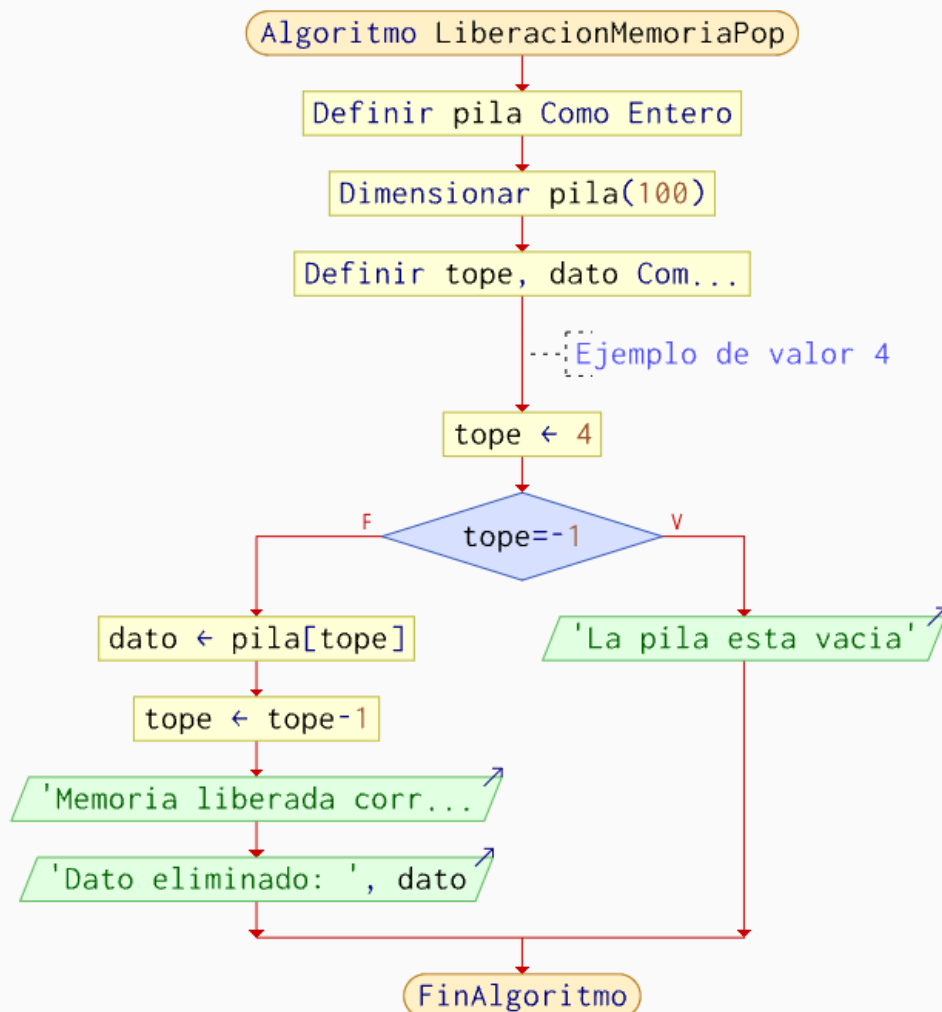
 tope <- tope - 1

 Escribir "Memoria liberada correctamente"

 Escribir "Dato eliminado: ", dato

FinSi

FinAlgoritmo



2.11. Algoritmo para visualizar la pila de memoria

Algoritmo VisualizarPilaMemoria

Definir pila[100] Como Entero

Definir tope, i Como Entero

Si tope = -1 Entonces

 Escribir "La pila está vacía"

Sino

 Escribir "Contenido actual de la pila:"

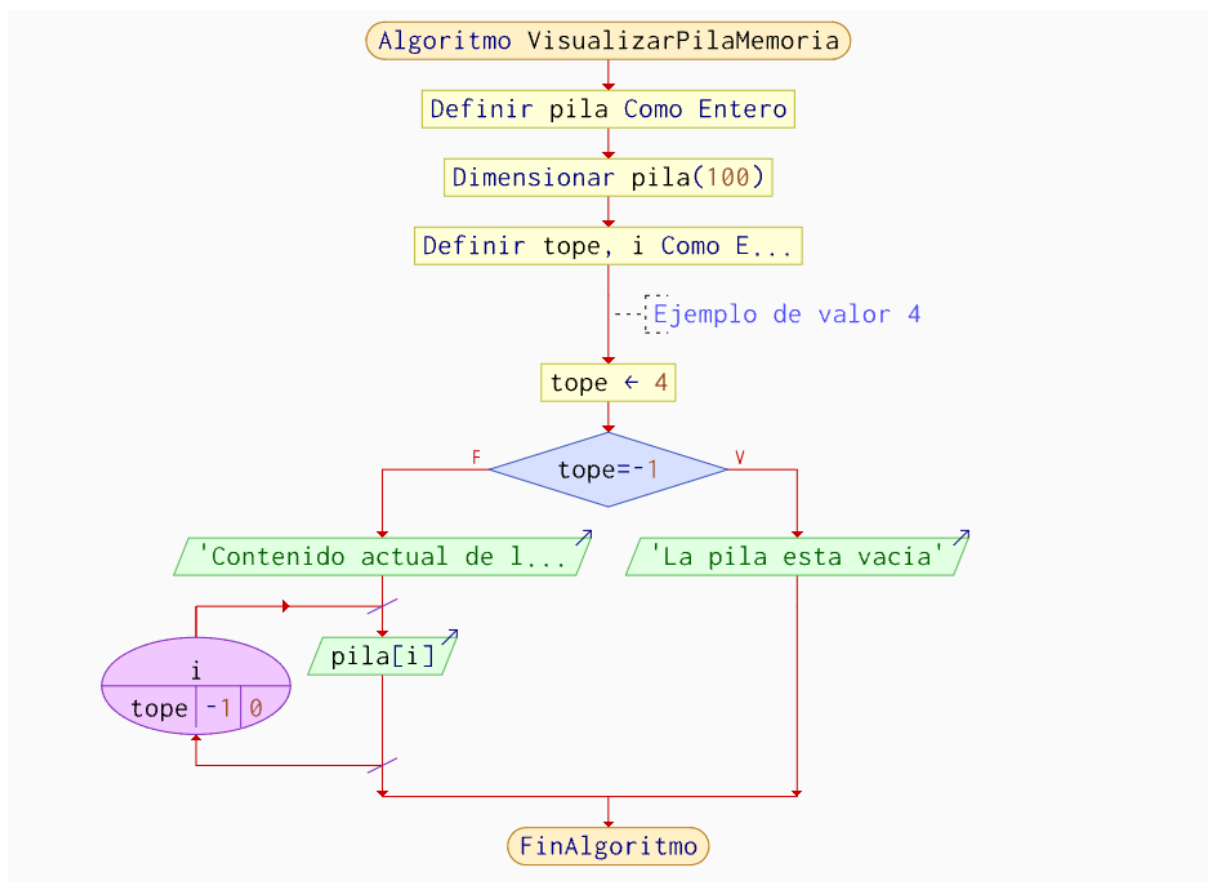
 Para i <- tope Hasta 0 Con Paso -1 Hacer

 Escribir pila[i]

 FinPara

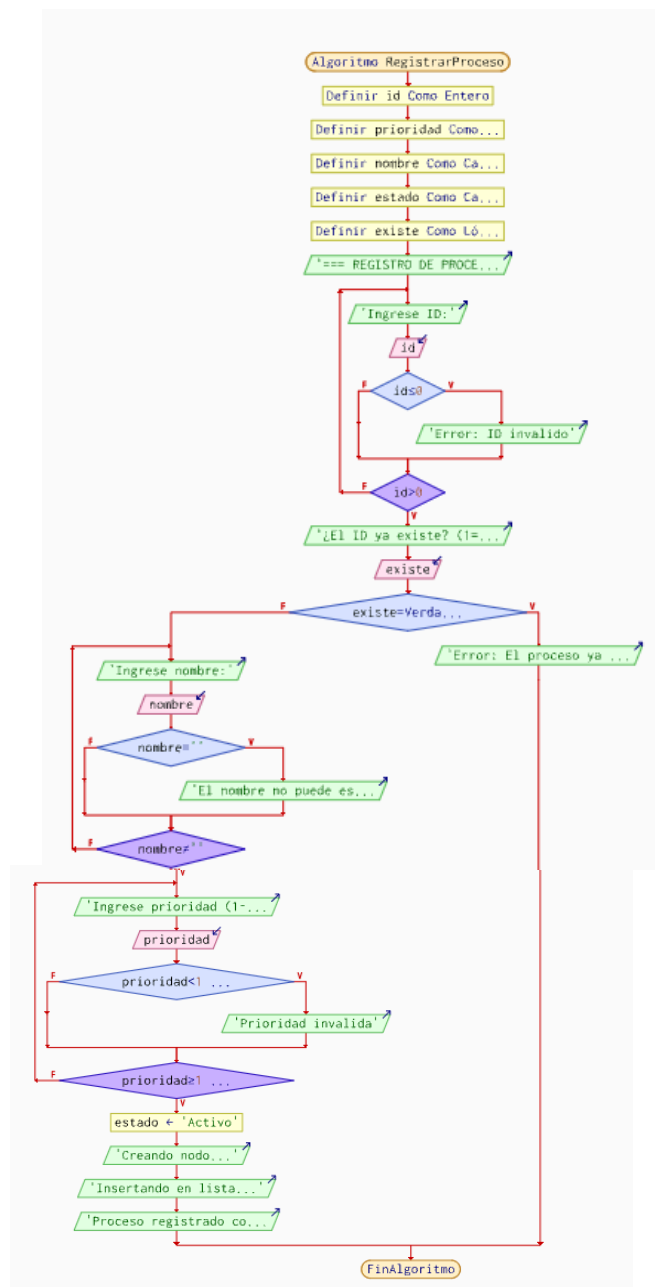
FinSi

FinAlgoritmo

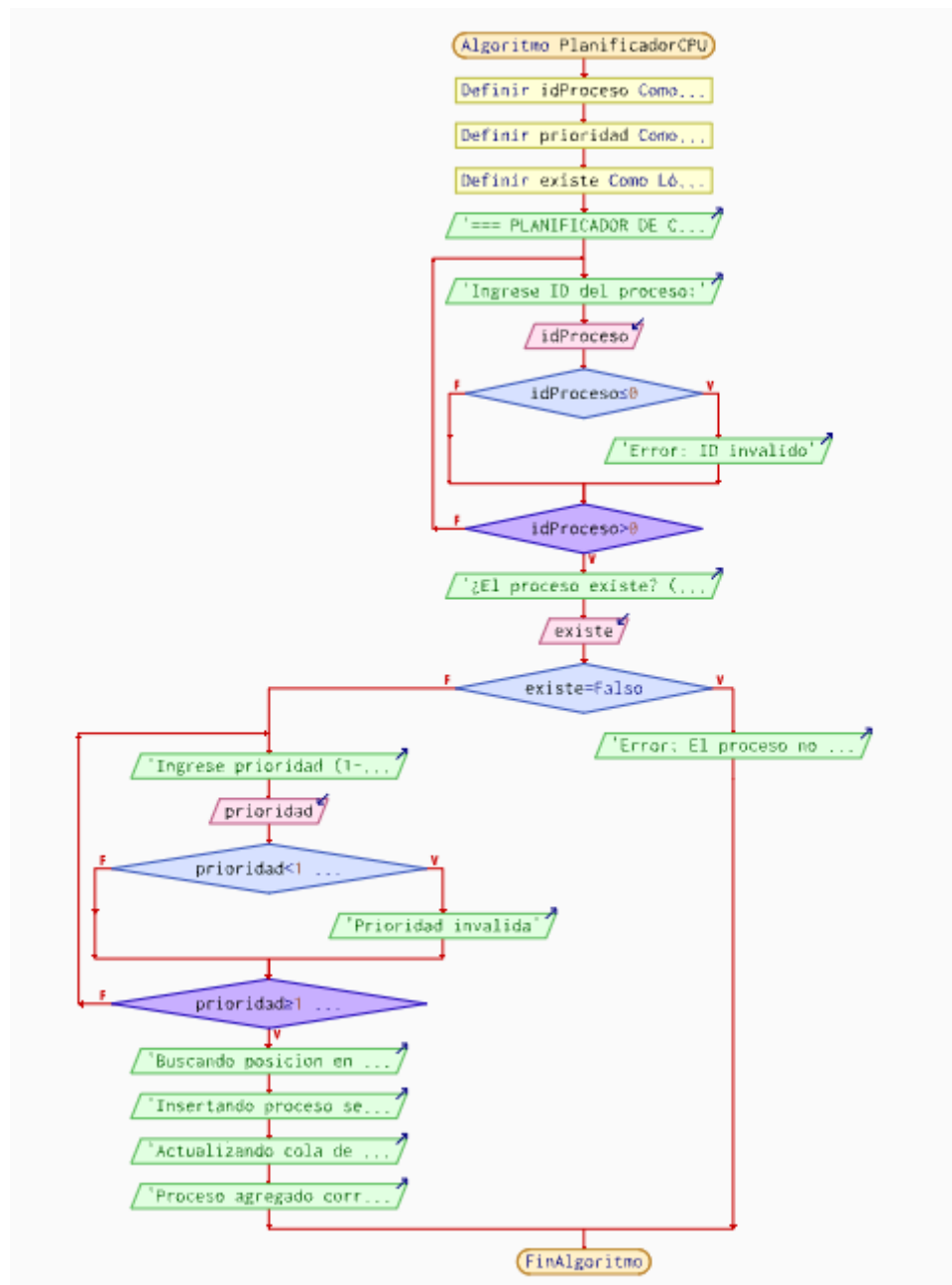


3. Diagramas de Flujo:

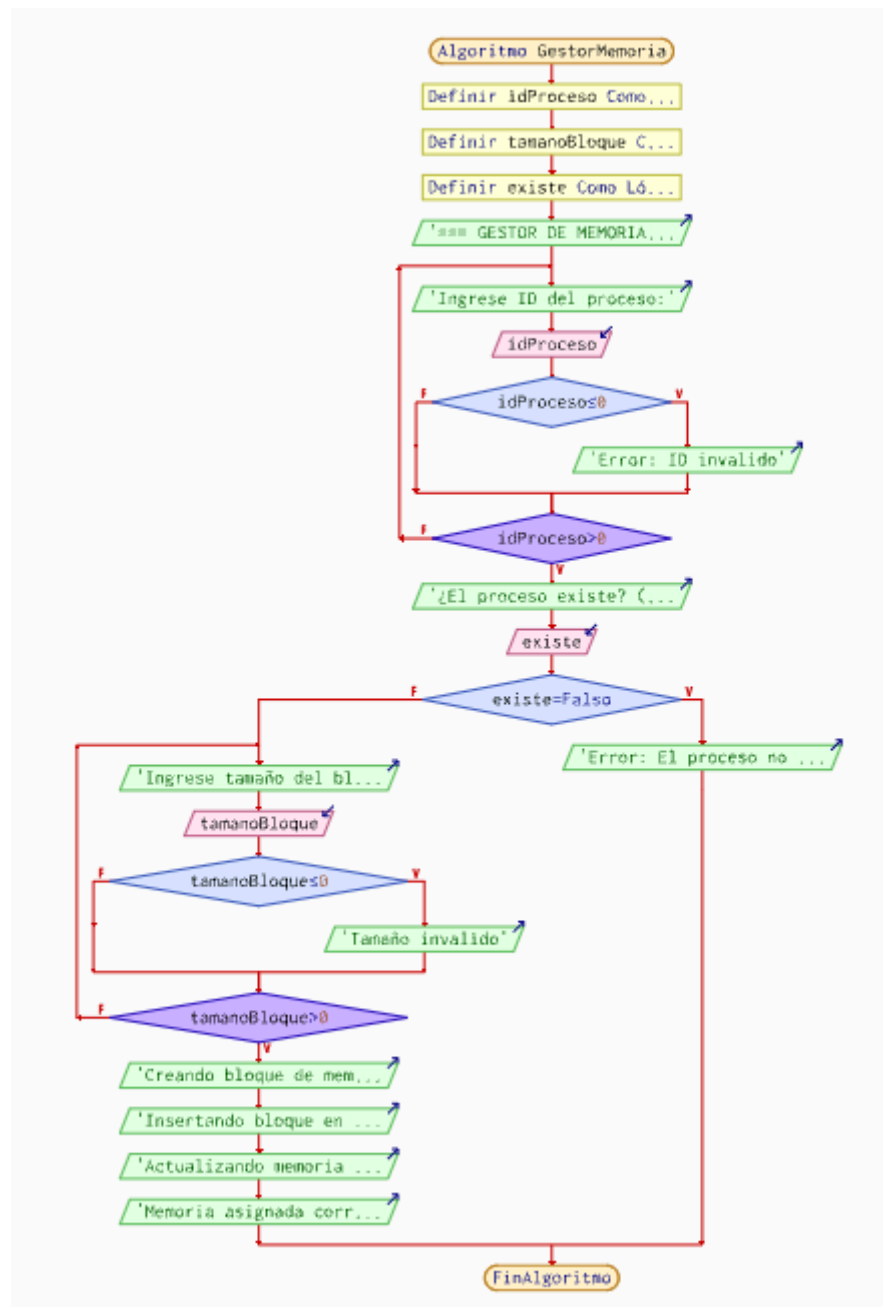
El diagrama de flujo que desarrollamos representa el funcionamiento de todo el Sistema de Gestión de Procesos mostrando la relación entre el menú principal y los diferentes módulos del sistema, como el Gestor de Procesos, Planificador de CPU y el Gestor de Memoria, cada módulo contiene sus propias operaciones y procesos.



GESTOR DE PROCESOS



PLANIFICADOR DE CPU



GESTOR DE MEMORIA

El flujo del sistema sigue una estructura ordenada y secuencial, el diagrama también incorpora bloques de decisión que validan lo ingresado por el usuario, para evitar errores en el funcionamiento y permitir regresar al menú principal cuando es necesario.

4. Justificación del diseño:

La organización modular permite distribuir responsabilidades específicas entre las distintas estructuras utilizadas por el sistema. Cada módulo se especializa en un conjunto determinado de operaciones, facilitando la administración de la información y mejorando la claridad del diseño general.

Los algoritmos fueron desarrollados utilizando estructuras condicionales y repetitivas para controlar el flujo de ejecución, validar información ingresada por el usuario y garantizar la integridad de los datos almacenados. Asimismo, se emplearon pseudocódigos y diagramas de flujo para representar visualmente la lógica de funcionamiento antes de su implementación definitiva en C++, permitiendo identificar posibles mejoras y corregir errores durante la etapa de diseño.

Capítulo 3

Solución Final

1. Código limpio, bien comentado y estructurado.

 CODIGO

2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación de datos

1. Registrar Proceso

```
===== SISTEMA DE GESTION DE PROCESOS =====  
1. Registrar proceso  
2. Mostrar procesos  
3. Buscar proceso  
4. Modificar prioridad  
5. Cambiar estado  
6. Eliminar proceso  
7. Encolar proceso en CPU  
8. Ejecutar proceso CPU  
9. Mostrar cola CPU  
10. Asignar memoria PUSH  
11. Liberar memoria POP  
12. Mostrar pila de memoria  
13. Mostrar tabla hash  
14. Guardar datos  
0. Salir  
Seleccione una opcion: 8  
  
La cola de CPU esta vacia.
```

El sistema solicita un ID, nombre y prioridad. Valida que el ID sea positivo y no esté repetido en la lista. Si es válido, crea un nuevo nodo con el estado "Activo", lo añade al final de la lista lineal y lo vincula a la Tabla Hash. Si falla alguna validación, la operación se cancela.

2. Mostrar Proceso

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

La cola de CPU esta vacia.
```

Revisa si hay procesos guardados en el sistema. Si la lista está vacía, te avisa; si no, recorre toda la lista desde el principio y te muestra en pantalla los datos de cada proceso en orden.

3. Buscar Proceso

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

La cola de CPU esta vacia.
```

Te pide el ID que quieres encontrar. Primero va directo a la tabla Hash para buscarlo al instante; si no lo encuentra ahí por algún choque de datos, busca uno por uno en la lista principal hasta dar con él o avisarte que no existe.

4. Modificar prioridad

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

La cola de CPU esta vacia.
```

Busca un proceso por su ID. Si existe, te permite escribirle una nueva prioridad, asegurándose de que esté en el rango del 1 al 5. Si no pones un número válido o el proceso no existe, cancela el cambio.

5. Cambiar Estado

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

La cola de CPU esta vacia.
```

El sistema solicita el ID de un proceso para buscarlo en la lista lineal. Si es localizado, limpia el búfer de entrada y permite al usuario escribir una nueva cadena de texto para actualizar el campo de estado (por ejemplo: "Bloqueado", "Suspendido", etc.). Si el ID no existe, muestra un mensaje de error.

6. Eliminar Proceso

```

===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

La cola de CPU esta vacia.

```

El sistema solicita el ID del proceso a borrar. Recorre la lista manteniendo un puntero al nodo anterior para no romper los enlaces. Al encontrarlo, realiza un puente en los punteros de la lista, elimina de forma segura su rastro en la Tabla Hash (colocando la posición en **NULL**) y libera el espacio físico del nodo en la memoria *Heap* mediante la instrucción **delete**.

7. Encolar proceso en CPU:

```

===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 7

Ingrese ID del proceso para enviar a CPU: 122
Proceso no encontrado.

```

El sistema valida que el ID ingresado exista previamente en la lista de procesos. Si el proceso no se encuentra registrado, se muestra el mensaje "Proceso no encontrado" y la operación de encolado es cancelada.

8. Ejecutar proceso CPU:

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8
La cola de CPU esta vacia.
```

El sistema verifica si la cola de CPU contiene procesos pendientes. Cuando la cola está vacía, muestra el mensaje "La cola de CPU está vacía" e impide la ejecución de procesos inexistentes.

9. Mostrar cola CPU:

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 9
La cola de CPU esta vacia.
```

El sistema verifica si existen procesos registrados en la cola de CPU antes de mostrarlos. Cuando la cola está vacía, se informa al usuario mediante el mensaje "La cola de CPU está vacía".

10. Asignar memoria PUSH

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 10

Ingrese ID del proceso: 233
Error: el proceso no existe.
```

El sistema valida que el ID del proceso exista antes de realizar la operación PUSH. Si el proceso no se encuentra registrado, no se asigna memoria y se muestra un mensaje de error, evitando almacenar bloques de memoria asociados a procesos inexistentes.

11. Liberar memoria POP

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 11

La pila de memoria esta vacia.
```

El sistema verifica que existan bloques de memoria almacenados en la pila antes de realizar la operación POP. Si no hay memoria asignada, la liberación es cancelada para evitar errores y se muestra un mensaje informando que la pila está vacía.

12. Mostrar pila de memoria

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 12

No hay memoria asignada en la pila.
```

El sistema verifica si existen bloques de memoria almacenados en la pila antes de mostrarlos. Cuando no hay memoria asignada, se informa al usuario mediante el mensaje "No hay memoria asignada en la pila".

13. Mostrar tabla hash

```
Seleccione una opcion: 13

=== TABLA HASH DE PROCESOS ===
Posicion [0]: Vacio
Posicion [1]: Vacio
Posicion [2]: Vacio
Posicion [3]: Vacio
Posicion [4]: Vacio
Posicion [5]: Vacio
Posicion [6]: Vacio
Posicion [7]: Vacio
Posicion [8]: Vacio
Posicion [9]: Vacio
```

El sistema valida si existen procesos almacenados en la tabla hash antes de mostrarlos. Cuando no hay procesos registrados, todas las posiciones aparecen vacías, indicando que la estructura no contiene información.

14. Guardar datos:

```
===== SISTEMA DE GESTION DE PROCESOS =====  
1. Registrar proceso  
2. Mostrar procesos  
3. Buscar proceso  
4. Modificar prioridad  
5. Cambiar estado  
6. Eliminar proceso  
7. Encolar proceso en CPU  
8. Ejecutar proceso CPU  
9. Mostrar cola CPU  
10. Asignar memoria PUSH  
11. Liberar memoria POP  
12. Mostrar pila de memoria  
13. Mostrar tabla hash  
14. Guardar datos  
0. Salir  
Seleccione una opcion: 14  
Datos guardados en procesos.txt
```

El sistema almacena la información de los procesos registrados en el archivo "procesos.txt", permitiendo conservar los datos para futuras ejecuciones del programa.

3. Manual de usuario

Al iniciar el programa el sistema carga automáticamente los procesos guardados si es que ya existen, a la vez también aparece el menú principal.

```
===== SISTEMA DE GESTION DE PROCESOS =====  
1. Registrar proceso  
2. Mostrar procesos  
3. Buscar proceso  
4. Modificar prioridad  
5. Cambiar estado  
6. Eliminar proceso  
7. Encolar proceso en CPU  
8. Ejecutar proceso CPU  
9. Mostrar cola CPU  
10. Asignar memoria PUSH  
11. Liberar memoria POP  
12. Mostrar pila de memoria  
13. Mostrar tabla hash  
14. Guardar datos  
0. Salir  
Seleccione una opcion: |
```

Introducción: El presente sistema permite gestionar procesos de manera eficiente mediante el uso de estructuras de datos dinámicas como listas enlazadas, colas de prioridad, pilas y tablas hash.

El usuario podrá registrar, buscar, modificar, eliminar procesos, así como simular la ejecución en CPU y la asignación de memoria.

3.1. Gestión de Procesos (Opciones 1 a 6)

1. Registrar proceso:

- ¿Cómo usarlo? Te pedirá un ID (número positivo único), un Nombre (ej. Chrome, Discord) y una Prioridad (del 1 al 5, donde 5 es lo más urgente).

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 1

Ingrese ID del proceso: 10
Ingrese nombre del proceso: Chrome
Ingrese prioridad (1-5): 3
Proceso registrado correctamente.

===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 1

Ingrese ID del proceso: 20
Ingrese nombre del proceso: Discord
```

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 1

Ingrese ID del proceso: 30
Ingrese nombre del proceso: VSCode
Ingrese prioridad (1-5): 5
Proceso registrado correctamente.

===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 1

Ingrese ID del proceso: 40
Ingrese nombre del proceso: Spotify
Ingrese prioridad (1-5): 4
Proceso registrado correctamente.
```

- **Nota:** Por defecto, el estado inicial de cualquier proceso nuevo será "Activo".

2. Mostrar procesos:

- *¿Cómo usarlo?* Despliega una lista en pantalla con todos los procesos registrados actualmente, mostrando su ID, Nombre, Prioridad y Estado.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 2

=== LISTA DE PROCESOS ===
ID: 10 | Nombre: Chrome | Prioridad: 3 | Estado: Activo
ID: 20 | Nombre: Discord | Prioridad: 1 | Estado: Activo
ID: 30 | Nombre: VSCode | Prioridad: 5 | Estado: Activo
ID: 40 | Nombre: Spotify | Prioridad: 4 | Estado: Activo
```

3. Buscar proceso:

- *¿Cómo usarlo?* Digita el ID del proceso. El programa lo buscará súper rápido usando la Tabla Hash; si hay algún problema, hará una búsqueda tradicional en la lista.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 3

Ingrese ID a buscar: 20

Proceso encontrado (via Lista):
ID: 20
Nombre: Discord
```

4. Modificar prioridad:

- *¿Cómo usarlo?* Si un proceso cambió de importancia, pon su ID y asígnale una nueva prioridad entre 1 y 5.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 4

Ingrese ID del proceso: 10
Ingrese nueva prioridad (1-5): 4
Prioridad actualizada correctamente.
```

5. Cambiar estado:

- *¿Cómo usarlo?* Ingresa el ID y escribe manualmente el nuevo estado del proceso (ej. Suspendido, Esperando un archivo).

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 5

Ingrese ID del proceso: 30
Ingrese nuevo estado: Suspendido
Estado actualizado correctamente.
```

6. Eliminar proceso:

- *¿Cómo usarlo?* Pon el ID del proceso que deseas borrar para eliminarlo por completo del sistema y liberar espacio en la memoria interna.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 6

Ingrese ID del proceso a eliminar: 20
Proceso eliminado correctamente.
```

3.2. Planificación de CPU (Opciones 7 a 9)

¡Ojo! Aquí importa la prioridad. Este módulo simula cómo el procesador decide qué tarea atender primero.

7. Encolar proceso en CPU:

- *¿Cómo usarlo?* Ingresa el ID de un proceso ya registrado. El programa creará una copia y la meterá a la fila de espera de la CPU.
- *¿Cómo se ordena?* Automáticamente pone hasta adelante los procesos con prioridad 5 y al final los de prioridad 1. Su estado cambiará a "En cola CPU".

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 7

Ingrese ID del proceso para enviar a CPU: 20
Proceso encolado segun prioridad en CPU.
```

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
```

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 7

Ingrese ID del proceso para enviar a CPU: 40
Proceso encolado segun prioridad en CPU.
```

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
```

8. Ejecutar proceso CPU:

- *¿Cómo usarlo?* No te pide ID. Al elegir esta opción, la CPU toma automáticamente el proceso al frente de la cola (el de mayor prioridad), lo procesa, actualiza su estado en la lista general a "Terminado/Activo" y lo saca de la fila.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 8

Ejecutando proceso en CPU:
ID: 30 | Nombre: VSCode | Prioridad: 5
Proceso ejecutado y retirado de la cola de CPU.
```

9. Mostrar cola CPU:

- *¿Cómo usarlo?* Te muestra visualmente el orden exacto en el que la CPU va a ir atendiendo los procesos que mandaste en la opción 7.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 9

=== COLA DE PRIORIDAD CPU ===
ID: 40 | Nombre: Spotify | Prioridad: 4
ID: 40 | Nombre: Spotify | Prioridad: 4
ID: 20 | Nombre: Discord | Prioridad: 1
```

3.3. Control de Memoria RAM (Opciones 10 a 12)

Regla LIFO: Funciona bajo una estructura de Pila (Stack). El último bloque en entrar es obligatoriamente el primero en salir.

10. Asignar memoria PUSH:

- *¿Cómo usarlo?* Ingresa el ID de un proceso existente, un número de bloque (ej. 102) y el tamaño en MB o KB (ej. 512). Este bloque se coloca al tope (arriba) de la pila.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 10

Ingrese ID del proceso: 10
Ingrese numero de bloque: 101
Ingrese tamano del bloque: 256
Memoria asignada correctamente con PUSH.
```

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 10

Ingrese ID del proceso: 40
Ingrese numero de bloque: 102
Ingrese tamano del bloque: 512
Memoria asignada correctamente con PUSH.
```

11. Liberar memoria POP:

- *¿Cómo usarlo?* No te pide ID. Al activarlo, eliminará el último bloque de memoria que hayas agregado (el que está hasta arriba) y te mostrará los datos del bloque destruido.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 11

Liberando memoria (POP):
ID Proceso: 10 | Bloque: 103 | Tamano: 128
Memoria liberada correctamente.
```


12. Mostrar pila de memoria:

- *¿Cómo usarlo?* Te muestra todos los bloques de memoria asignados, ordenados desde el último que entró (arriba) hasta el primero (abajo).

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 12

=== PILA DE MEMORIA ===
ID Proceso: 10 | Bloque: 103 | Tamano: 128
ID Proceso: 40 | Bloque: 102 | Tamano: 512
ID Proceso: 10 | Bloque: 101 | Tamano: 256
```

3.4. Almacenamiento y Sistema (Opciones 13, 14 y 0)

13. Mostrar tabla hash:

- *¿Cómo usarlo?* Herramienta técnica. Muestra las 10 posiciones del "servidor interno" de búsqueda rápida y qué proceso está asignado a cada casilla.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 13

=== TABLA HASH DE PROCESOS ===
Posicion [0]: ID 40 - Spotify
Posicion [1]: Vacio
Posicion [2]: Vacio
Posicion [3]: Vacio
Posicion [4]: Vacio
Posicion [5]: Vacio
Posicion [6]: Vacio
Posicion [7]: Vacio
Posicion [8]: Vacio
Posicion [9]: Vacio
```

14. Guardar datos:

- *¿Cómo usarlo?* Guarda manualmente toda tu lista de procesos en el archivo `procesos.txt` para asegurar tu información ante apagones.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 14
Datos guardados en procesos.txt
```

0. Salir:

- *¿Cómo usarlo?* La forma correcta de cerrar. Al presionar 0, el sistema guarda los procesos automáticamente, limpia de forma segura la memoria RAM de tu computadora para no dejar basura digital, y cierra la consola.

```
===== SISTEMA DE GESTION DE PROCESOS =====
1. Registrar proceso
2. Mostrar procesos
3. Buscar proceso
4. Modificar prioridad
5. Cambiar estado
6. Eliminar proceso
7. Encolar proceso en CPU
8. Ejecutar proceso CPU
9. Mostrar cola CPU
10. Asignar memoria PUSH
11. Liberar memoria POP
12. Mostrar pila de memoria
13. Mostrar tabla hash
14. Guardar datos
0. Salir
Seleccione una opcion: 0
Datos guardados en procesos.txt
Memoria de estructuras liberada. Saliendo del sistema...

-----
Process exited after 2.677 seconds with return value 0
Presione una tecla para continuar . . .
```

3.5. Consejos Rápidos de Uso

- **Flujo lógico recomendado:** Primero **Registra** un proceso (Opción 1), luego **Encolalo** en la CPU (Opción 7), revisa la **Fila** (Opción 9) y finalmente **Ejecútalo** (Opción 8).
- **Cuidado con la memoria:** No intentes usar la Opción 10 (Asignar memoria) con un ID inventado; primero debes haber registrado ese ID con la Opción 1.
- **Cierre Limpio:** Intenta salir siempre con la Opción 0 para asegurar el correcto guardado de tus datos.

Capítulo 4

Evidencias de trabajo
colaborativo

1. Repositorio y control de versiones

1.1. Registro de commits

Commits		
main	All users	All time
Commits on Jun 3, 2026		
Añadi el codigo	Fab070520 committed 1 minute ago	5ba6393
Subí el acta correspondiente a la 7ª reunión	DanielCelis2121 authored 6 minutes ago	50c052
Subí el acta correspondiente a la 6ª reunión	DanielCelis2121 authored 9 minutes ago	3d7be18
Commits on May 27, 2026		
Se agrego la justificación del diseño.	MiguelAngel-art authored last week	751798
Subí el acta correspondiente a la la 5ª reunión.	DanielCelis2121 authored last week	36f2af8
Se agrego el diagrama de flujo	Joseph20175 authored last week	5cd23d2
Subí el acta correspondiente a la la 4ª reunión.	MiguelAngel-art authored last week	242350d
Delete Actas/Acta V1.3.pdf	MiguelAngel-art authored last week	30elcct
Add files via upload	MiguelAngel-art authored last week	70a80b3
Se agrego los pseudocódigos principales.	DanielCelis2121 authored last week	a8f62d4
Se agrego descripción de estructuras de datos	Fab072014 authored last week	bdda298

Subí el acta correspondiente a la 3ª reunión.	MiguelAngel-art authored last week	ceaab7b
Commits on May 24, 2026		
Se subió la versión final del capítulo 1	Joseph20175 authored last week	34bbd87
Se corrigió las justificaciones de la elecciones del caso.	MiguelAngel-art authored last week	309b9ad
Eliminé un archivo erróneo	Fab072014 authored last week	1b03251
Se corrigió y mejoro las estructuras de datos propuestas	Joseph20175 authored last week	5eda993
Corrección de requerimientos.	DanielCelis2121 authored last week	3987b88
Corrección de los requerimientos.	DanielCelis2121 authored last week	caeb9e9
Se corrigió y mejoró el README del proyecto	Fab072014 authored last week	3838bbb
Se corrigió y mejoró la descripción del problema	Fab072014 authored last week	486b9cb
Commits on May 18, 2026		
Subí el acta correspondiente a la 2ª reunión	Joseph20175 authored 2 weeks ago	b7476fe
Subí el acta correspondiente a la 1ª reunión	Joseph20175 authored 2 weeks ago	35d1db6
Commits on May 17, 2026		

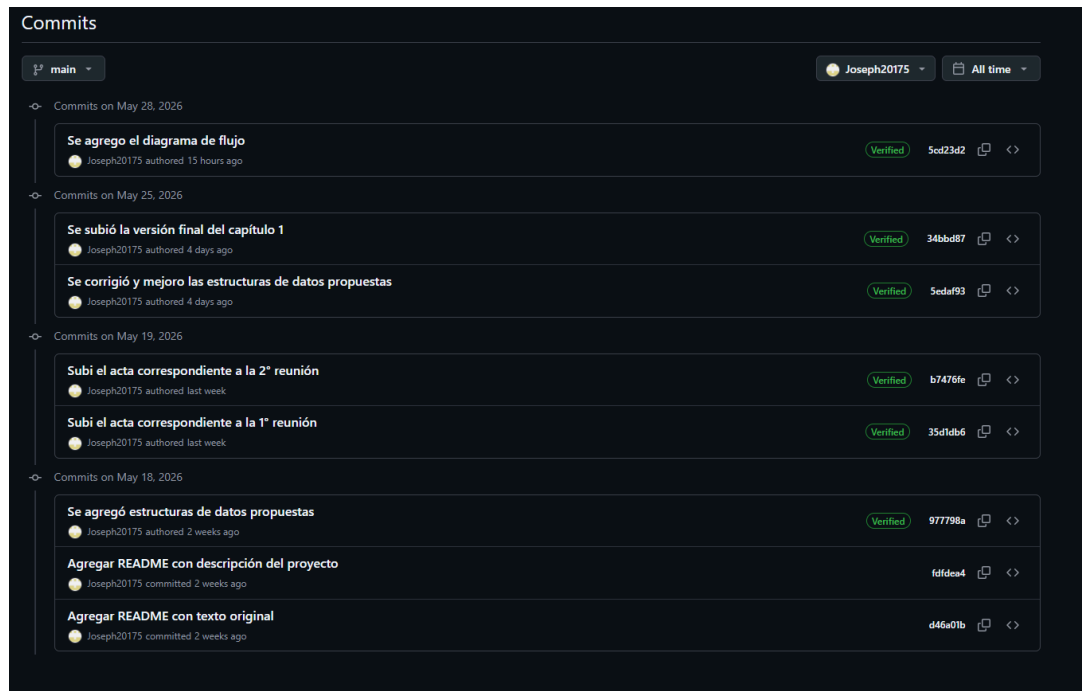
Commits on May 17, 2026		
Se agregaron las justificaciones de las elecciones del caso	MiguelAngel-art authored 2 weeks ago	beef3d6
Se agrego estructuras de datos propuestas	Joseph20175 authored 2 weeks ago	977798a
Agregar README con descripción del proyecto	Joseph20175 committed 2 weeks ago	fd1dc44
Agregar README con texto original	Joseph20175 committed 2 weeks ago	d46a01b
Se agrego los requerimientos del caso	DanielCelis2121 authored 2 weeks ago	e15f609
Realicé descripción del problema	Fab072014 authored 2 weeks ago	78e6214
Estructura inicial: Carpetas Vinformes y Actas	Fab070520 committed 2 weeks ago	f46488d

1.2. Historial de ramas

En el proyecto decidimos usar solamente la rama principal (main). La razón fue que el proyecto era simple y el equipo era pequeño. Esto permitirá trabajar de una manera más sencilla.

1.3. Evidencia por cada integrante del equipo

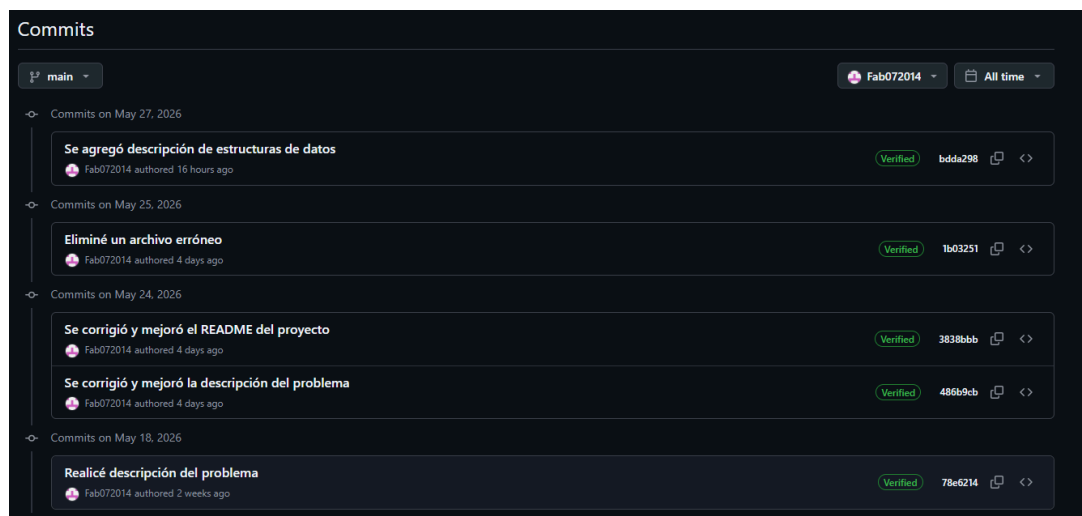
1.3.1. Malpartida Arroyo Joseph Rafael



This screenshot shows the commit history for user Joseph20175 on the main branch. The commits are grouped by date, showing a series of updates to the project structure and documentation.

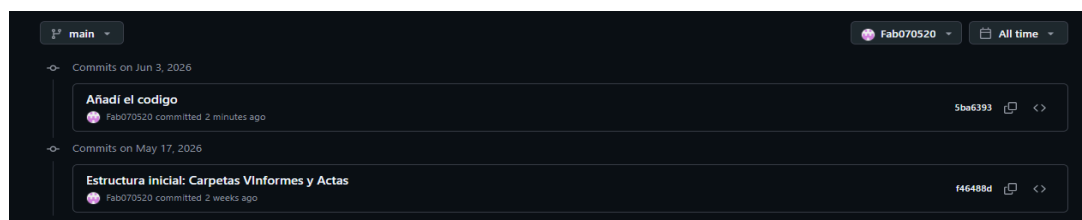
Commit Message	Author	Time	Hash
Se agrego el diagrama de flujo	Joseph20175	15 hours ago	5cd23d2
Se subió la versión final del capítulo 1	Joseph20175	4 days ago	34b8d87
Se corrigió y mejoro las estructuras de datos propuestas	Joseph20175	4 days ago	5eda193
Subi el acta correspondiente a la 2ª reunión	Joseph20175	last week	b7476fe
Subi el acta correspondiente a la 1ª reunión	Joseph20175	last week	35d1db6
Se agregó estructuras de datos propuestas	Joseph20175	2 weeks ago	977798a
Agregar README con descripción del proyecto	Joseph20175	2 weeks ago	fdfdea4
Agregar README con texto original	Joseph20175	2 weeks ago	d46a01b

1.3.2. Ñaupari Bonilla Miguel Fabricio



This screenshot shows the commit history for user Fab072014 on the main branch. The commits show updates to the data structure description, removal of an erroneous file, and improvements to the README and problem description.

Commit Message	Author	Time	Hash
Se agregó descripción de estructuras de datos	Fab072014	16 hours ago	bdda298
Eliminé un archivo erróneo	Fab072014	4 days ago	1b03251
Se corrigió y mejoró el README del proyecto	Fab072014	4 days ago	3838bbb
Se corrigió y mejoró la descripción del problema	Fab072014	4 days ago	486b9cb
Realicé descripción del problema	Fab072014	2 weeks ago	78e6214



This screenshot shows the commit history for user Fab070520 on the main branch. The commits show the addition of code and the initial project structure.

Commit Message	Author	Time	Hash
Añadi el codigo	Fab070520	2 minutes ago	5ba6393
Estructura Inicial: Carpetas Vinformes y Actas	Fab070520	2 weeks ago	f46488d

1.3.3. Celis Ricra Daniel Angel

main	DanielCelis2121	All time
Commits on Jun 3, 2026		
Subí el acta correspondiente a la 7ª reunión	Verified	50c052
Subí el acta correspondiente a la 6ª reunión	Verified	3d7bc18
Commits on May 27, 2026		
Subí el acta correspondiente a la la 5ª reunión.	Verified	36f2af8
Se agrego los pseudocódigos principales.	Verified	a8f62dd
Commits on May 24, 2026		
Corrección de requerimientos.	Verified	3987b88
Corrección de los requerimientos.	Verified	caeb9e9
Commits on May 17, 2026		
Se agrego los requerimientos del caso	Verified	ef5f609

1.3.4. Barrientos Chambero Miguel A

main	MiguelAngel-art	All time
Commits on May 28, 2026		
Se agrego la justificación del diseño.	Verified	f751798
Subí el acta correspondiente a la la 4ª reunión.	Verified	242350d
Delete Actas/Acta V1.3.pdf	Verified	30e1cef
Commits on May 27, 2026		
Add files via upload	Verified	70a80b3
Commits on May 25, 2026		
Subí el acta correspondiente a la 3ª reunión.	Verified	ceaab7b
Se corrigió las justificaciones de la elecciones del caso.	Verified	309bfad
Commits on May 18, 2026		
Se agregaron las justificaciones de las elecciones del caso	Verified	beef3d6

1.4. Enlace a la herramienta colaborativa

El siguiente enlace es el de nuestro repositorio:
<https://github.com/Joseph20175/Trabajo-Estructura-de-datos.git>

1.5. Enlace a las diapositivas de exposicion

El siguiente enlace es de nuestras diapositivas de exposición:
<https://canva.link/zqqe1olqn8rzt8k>

2. Plan de trabajo y roles + asignados

Al principio del proyecto todo el equipo se designaron tareas y responsabilidades para cada integrante, con el fin de trabajar de manera ordenada y avanzar correctamente en todas las etapas del desarrollo del sistema, además se pudo establecer un cronograma con fechas límites para la entrega de cada avance del trabajo, permitiendo controlar mejor el progreso del proyecto.

2.1. Documento inicial donde se asignan tareas y responsabilidades.

- **Malpartida Arroyo Joseph Rafael**
Encargado del desarrollo de la lista enlazada, además de las funciones relacionadas con el registro, búsqueda y eliminación de procesos.
- **Ñaupari Bonilla Miguel Fabricio**
Responsable de la implementación de la cola de prioridad y de la simulación del planificador de CPU dentro del sistema.
- **Celis Ricra Daniel Angel**
Encargado del desarrollo de la pila dinámica para la gestión de memoria y del manejo de persistencia de datos mediante archivos.
- **Barrientos Chambero Miguel Angel**
Responsable del diseño del menú interactivo, integración de módulos y validación general del funcionamiento del sistema.

2.2. Cronograma con fechas límite para cada entrega parcial.

Semana	Actividad	Descripción
Semana 10 18 de Mayo - 22 de Mayo	Capítulo 1: Análisis del problema	Se realizará la descripción del problema, los requerimientos del sistema, las estructuras de datos propuestas y la justificación de las estructuras utilizadas en el proyecto.
Semana 11 25 de Mayo - 29 de Mayo	Capítulo 2: Diseño de la solución	Se desarrollará el diseño del sistema, incluyendo las operaciones principales, pseudocódigos y diagramas de flujo necesarios para el funcionamiento del proyecto.
Semana 12 1 de Junio - 5 de Junio	Capítulo 3: Solución final	Se integrará el código completo del sistema, además se realizarán pruebas de funcionamiento, validación de datos y la elaboración del manual de usuario.

2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).

Las actas y registro de las reuniones realizadas en el equipo se encuentran en el siguiente documento: [Actas - Documentos de Google](#)